Tasks on Terraform-03

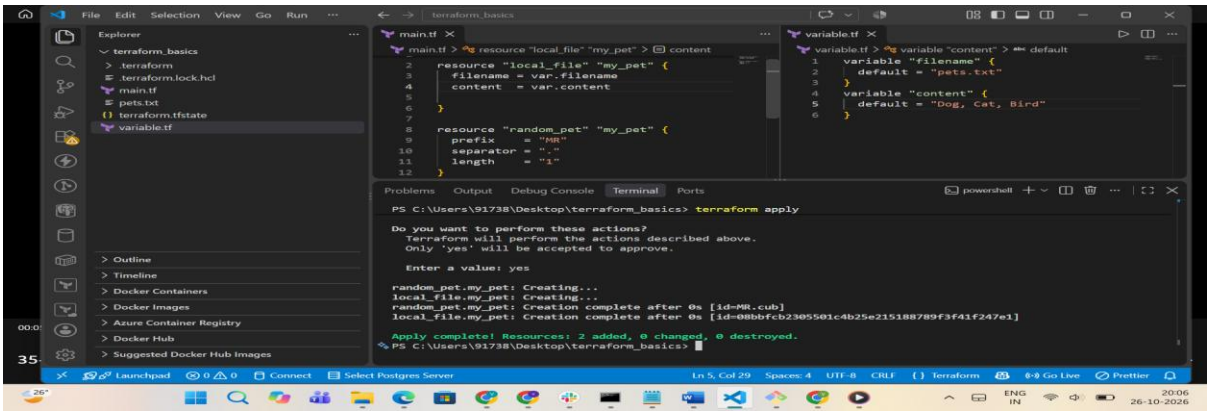
1. **Watch the Terraform-03 video.**

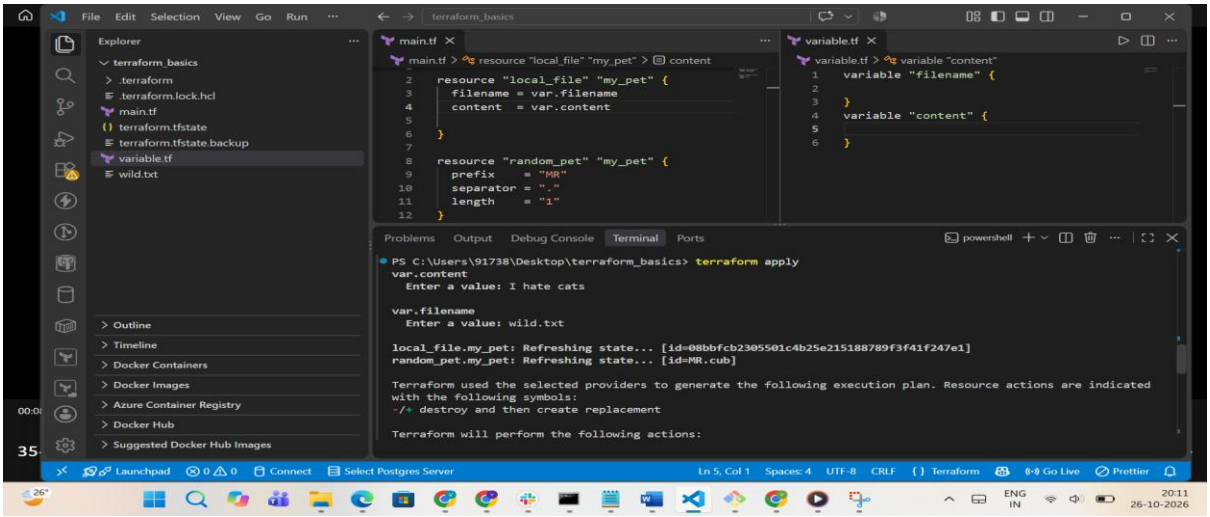
   Watching video of terraform-03 completed.

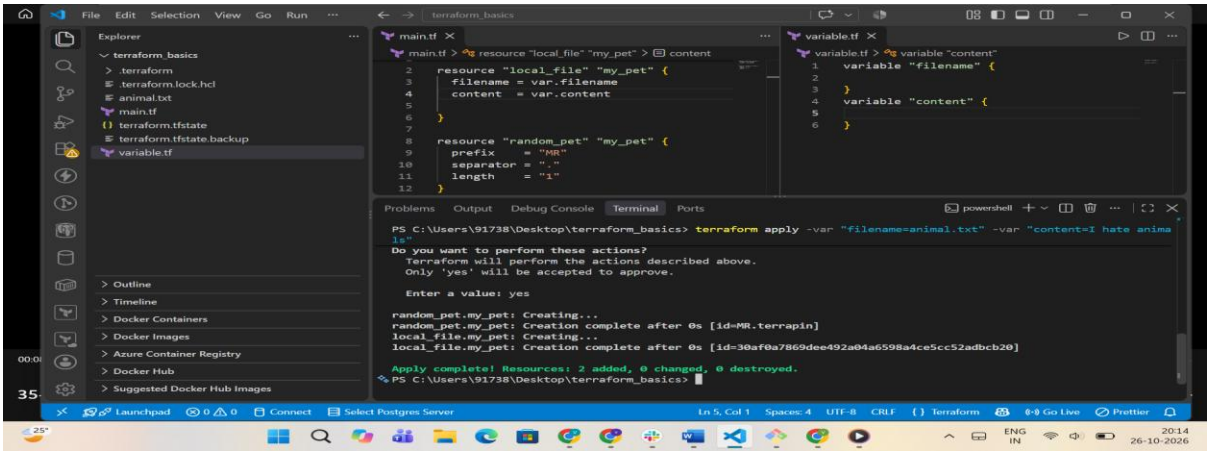
2. **Execute the script shown in the video.**

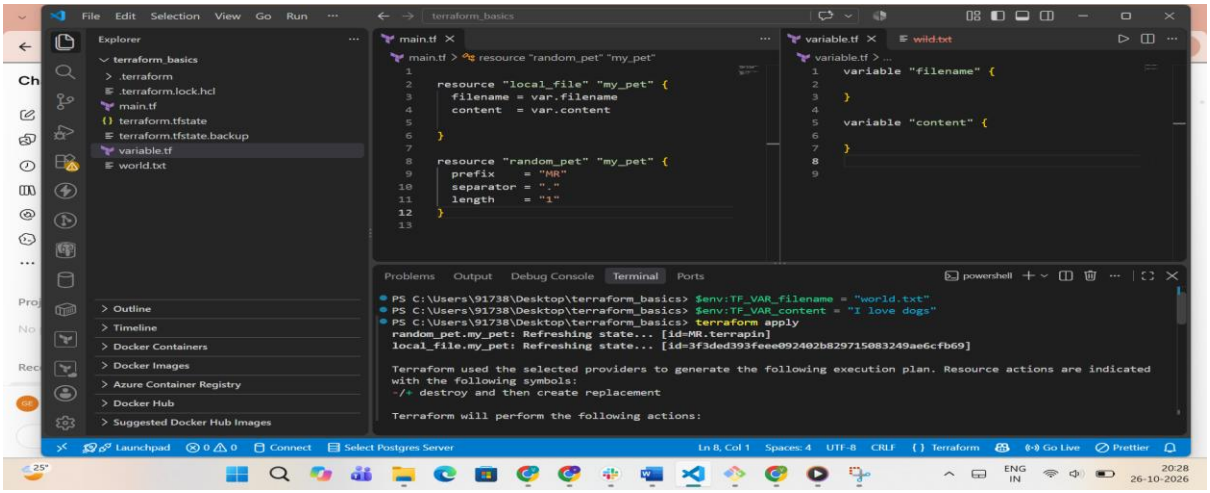
   Implementation steps:

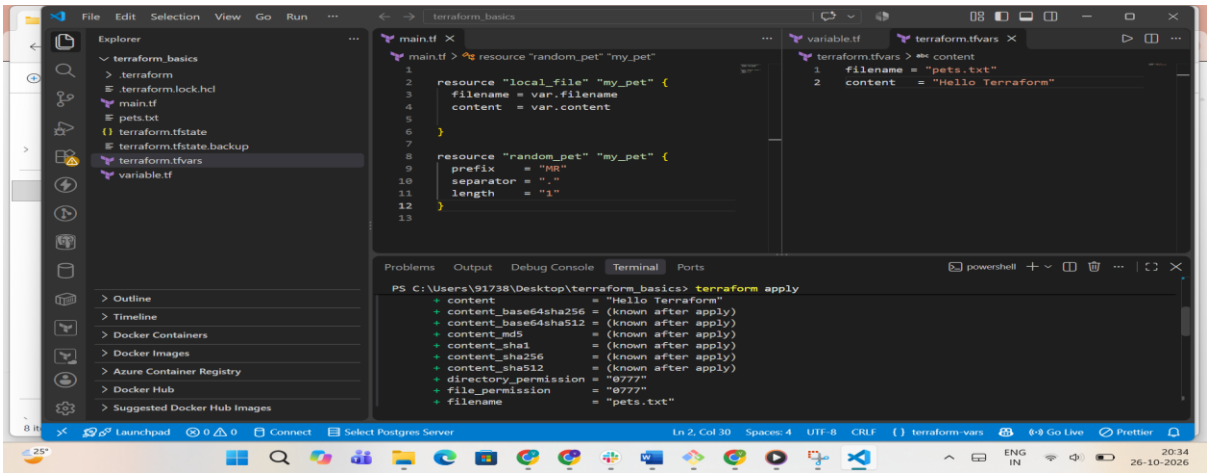
   1.Using variables in different ways:



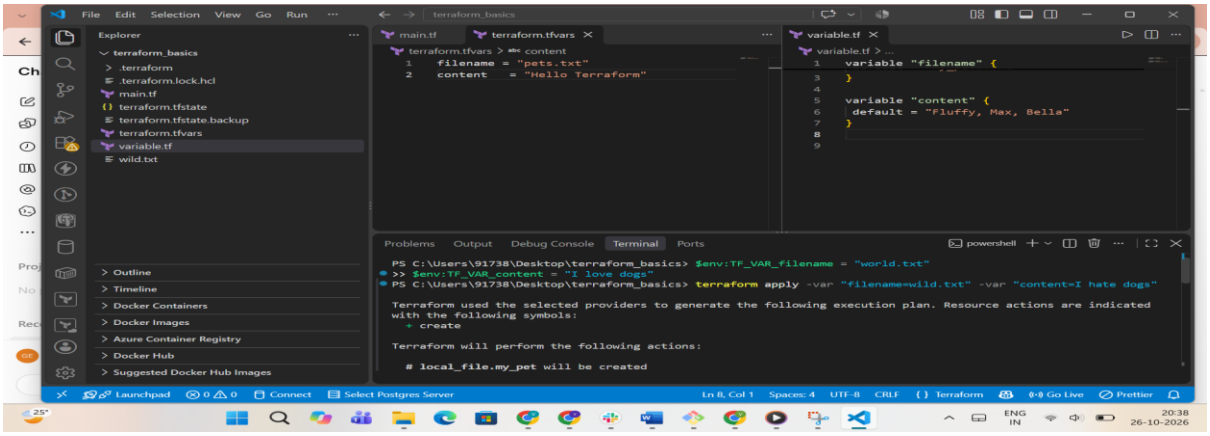



terraform_basics
Explorer
terraform_basics
.terraform
.terraform.lock.hcl
animal.txt
main.tf
terraform.tfstate
terraform.tfstate.backup
variable.tf
main.tf
variable.tf
resource "local_file" "my_pet" {
filename = var.filename
content = var.content
}
resource "random_pet" "my_pet" {
prefix = "MR"
separator = "."
length = "1"
}
variable "filename" {
}
variable "content" {
}
Problems Output Debug Console Terminal Ports
powershell
PS C:\Users\91738\Desktop\terraform_basics> terraform apply -var "filename=animal.txt" -var "content=I hate animals"
Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.
Enter a value: yes
random_pet.my_pet: Creating...
random_pet.my_pet: Creation complete after 0s [id=MR.terrapin]
local_file.my_pet: Creating...
local_file.my_pet: Creation complete after 0s [id=30af0a7869dee492a04a6598a4ce5cc52adbcb20]
Apply complete! Resources: 2 added, 0 changed, 0 destroyed.
PS C:\Users\91738\Desktop\terraform_basics>
Outline
Timeline
Docker Containers
Docker Images
Azure Container Registry
Docker Hub
Suggested Docker Hub Images
Launchpad
Connect
Select Postgres Server
Ln 5, Col 1
Spaces: 4
UTF-8
CRLF
Terraform
Go Live
Prettier
ENG
IN
20:14
26-10-2026

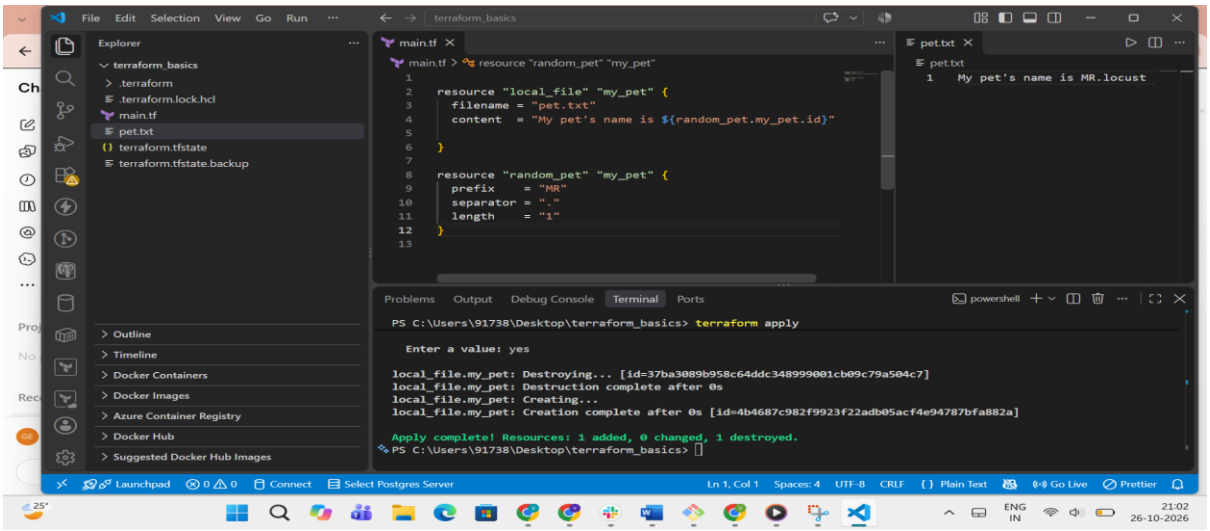

terraform_basics
Explorer
terraform_basics
.terraform
.terraform.lock.hcl
main.tf
terraform.tfstate
terraform.tfstate.backup
variable.tf
world.txt
main.tf
variable.tf
wild.txt
resource "local_file" "my_pet" {
filename = var.filename
content = var.content
}
resource "random_pet" "my_pet" {
prefix = "MR"
separator = "."
length = "1"
}
variable "filename" {
}
variable "content" {
}
Problems Output Debug Console Terminal Ports
powershell
PS C:\Users\91738\Desktop\terraform_basics> $env:TF_VAR_filename = "world.txt"
PS C:\Users\91738\Desktop\terraform_basics> $env:TF_VAR_content = "I love dogs"
PS C:\Users\91738\Desktop\terraform_basics> terraform apply
random_pet.my_pet: Refreshing state... [id=MR.terrapin]
local_file.my_pet: Refreshing state... [id=3f3ded393feee092402b829715083249ae6cfb69]
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement
Terraform will perform the following actions:
Outline
Timeline
Docker Containers
Docker Images
Azure Container Registry
Docker Hub
Suggested Docker Hub Images
Launchpad
Connect
Select Postgres Server
Ln 8, Col 1
Spaces: 4
UTF-8
CRLF
Terraform
Go Live
Prettier
ENG
IN
20:28
26-10-2026

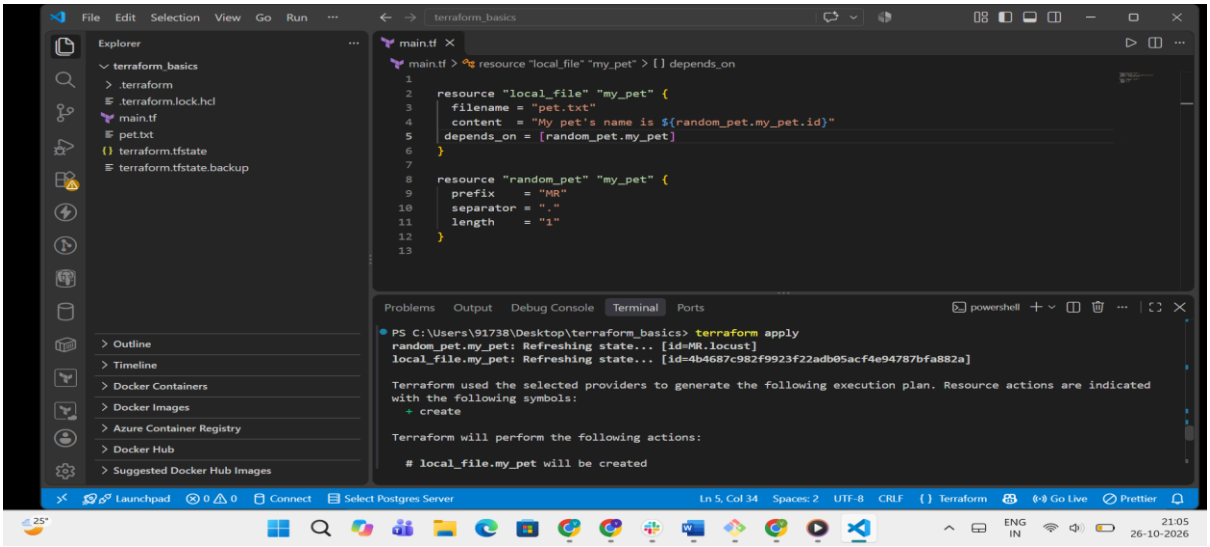

terraform_basics
Explorer
terraform_basics
.terraform
.terraform.lock.hcl
main.tf
pets.txt
terraform.tfstate
terraform.tfstate.backup
terraform.tfvars
variable.tf
main.tf
variable.tf
terraform.tfvars
resource "local_file" "my_pet" {
filename = var.filename
content = var.content
}
resource "random_pet" "my_pet" {
prefix = "MR"
separator = "."
length = "1"
}
filename = "pets.txt"
content = "Hello Terraform"
Problems Output Debug Console Terminal Ports
powershell
PS C:\Users\91738\Desktop\terraform_basics> terraform apply
+ content = "Hello Terraform"
+ content_base64sha256 = (known after apply)
+ content_base64sha512 = (known after apply)
+ content_md5 = (known after apply)
+ content_sha1 = (known after apply)
+ content_sha256 = (known after apply)
+ content_sha512 = (known after apply)
+ directory_permission = "0777"
+ file_permission = "0777"
+ filename = "pets.txt"
Outline
Timeline
Docker Containers
Docker Images
Azure Container Registry
Docker Hub
Suggested Docker Hub Images
Launchpad
Connect
Select Postgres Server
Ln 2, Col 30
Spaces: 4
UTF-8
CRLF
terraform-vars
Go Live
Prettier
ENG
IN
20:34
26-10-2026

2.Precedence order :

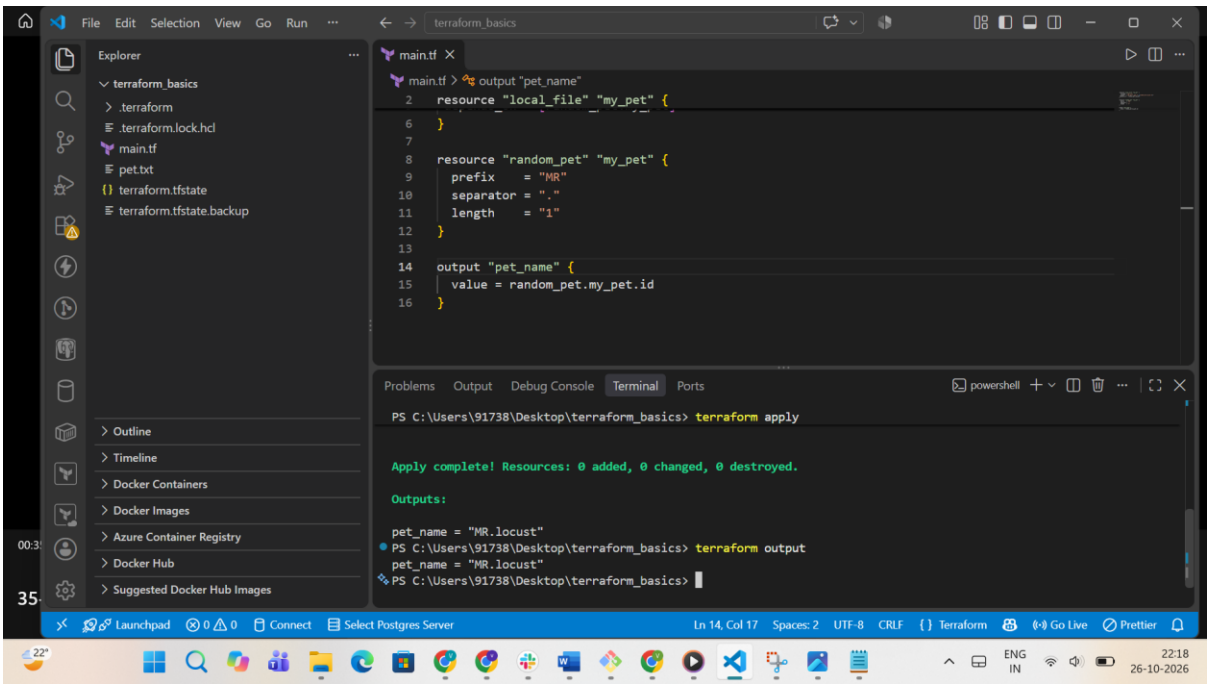


3. Resource and Dependencies:

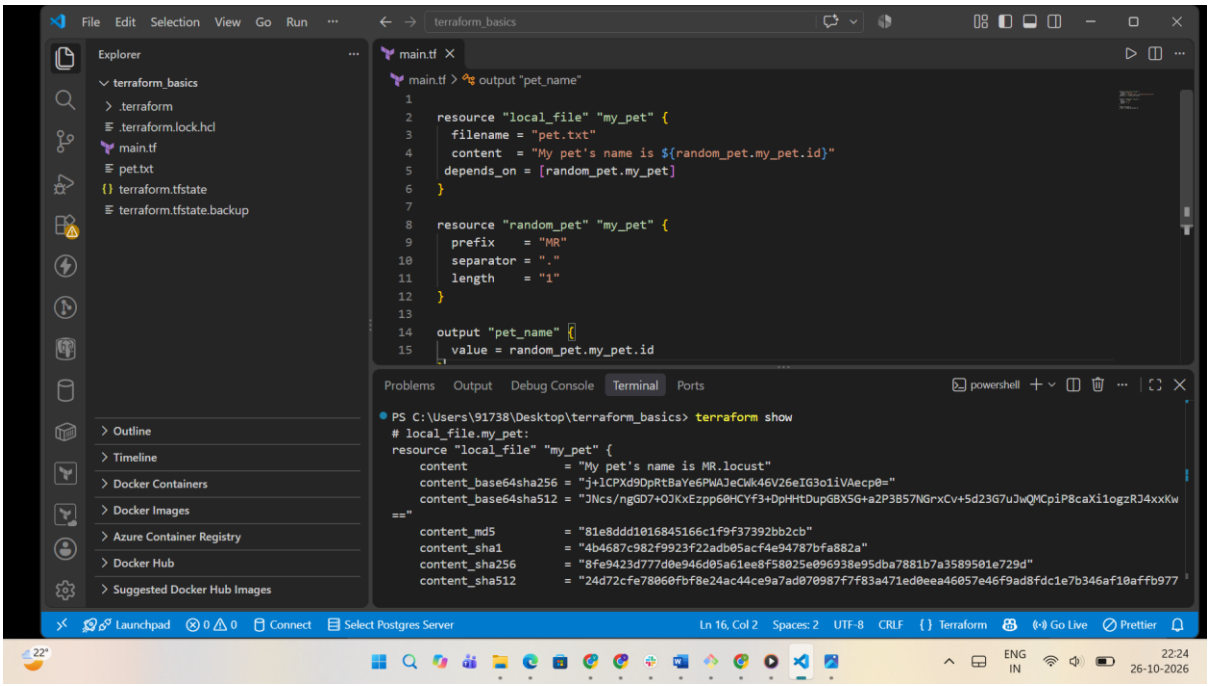




4. Output variables:



5.Terraform show:



3. Integrate Terraform in Jenkins using the Terraform plugin.

Implementations steps:

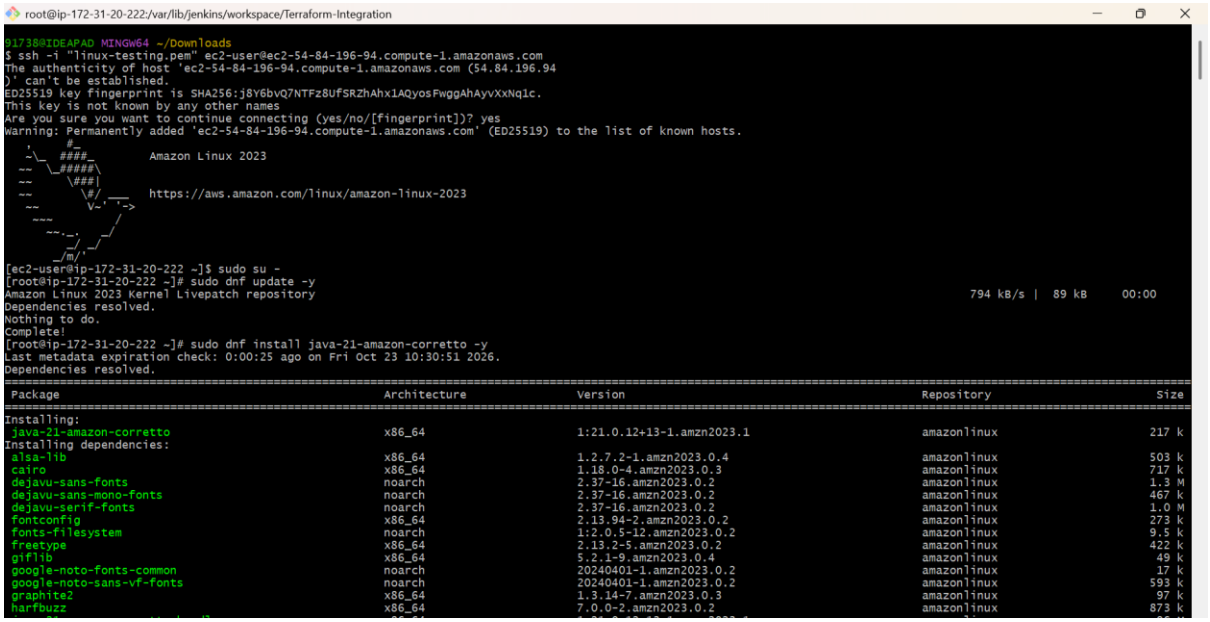
1. Launch an EC2 instance using Amazon Linux.
2. Connect to the EC2 instance through SSH.

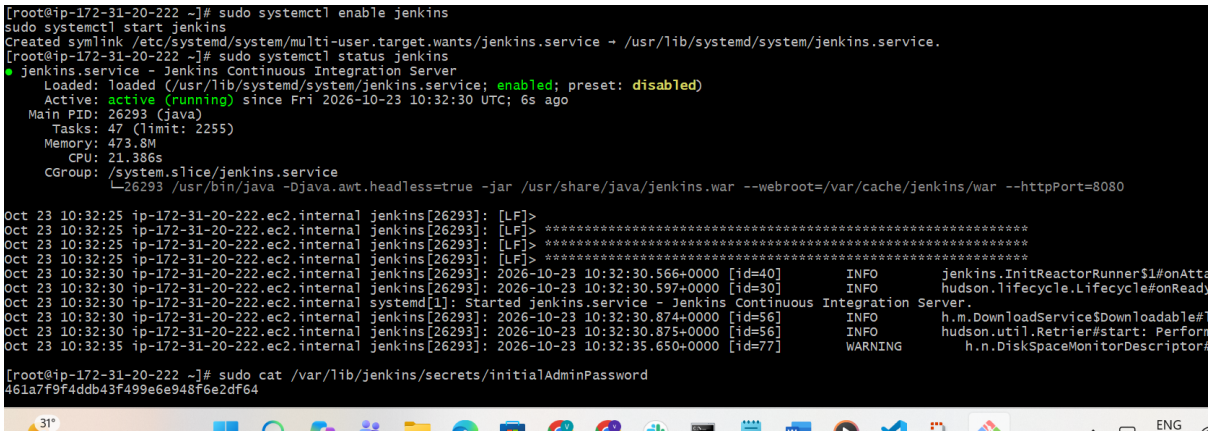
3. Install Java and Jenkins.
4. Start Jenkins and enable it to run automatically:

   sudo systemctl enable jenkins

   sudo systemctl start jenkins
5. Configure the EC2 security group to allow access to the Jenkins web interface.
6. Open Jenkins in a browser using the EC2 public IP and configured port.
7. Complete the initial Jenkins setup and log in as the administrator.
8. Navigate to Manage Jenkins → Plugins.
9. Install the Terraform plugin.
10. Install Terraform on the EC2 instance and verify the installation:

    /usr/bin/terraform
11. Navigate to Manage Jenkins → Tools and configure the Terraform installation.



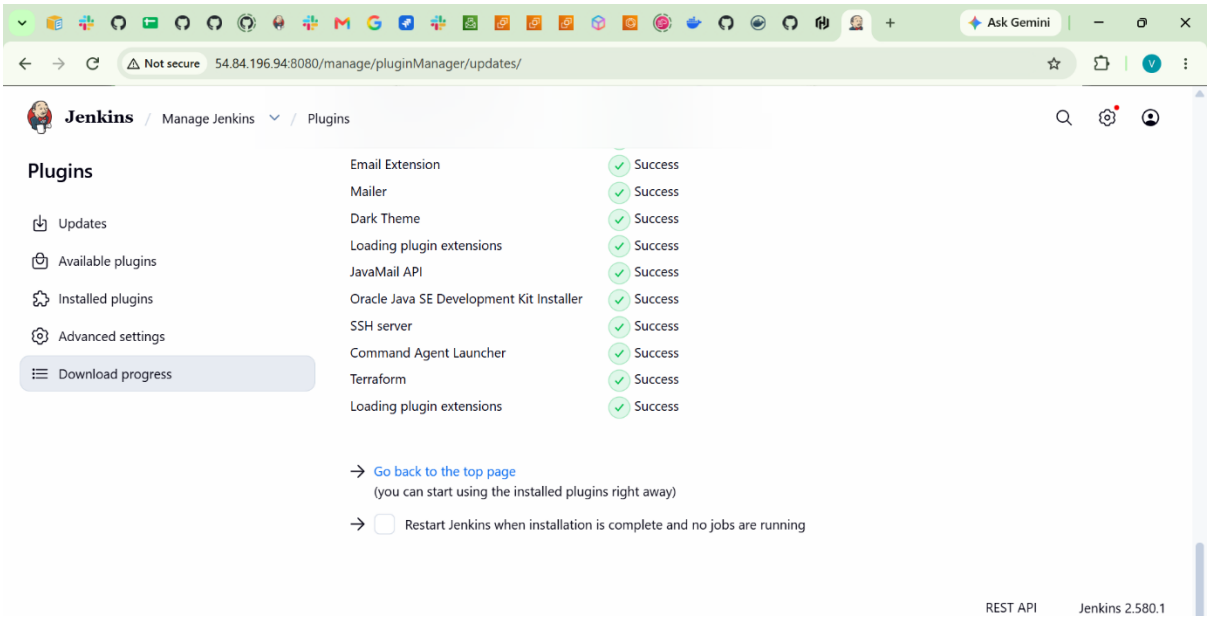


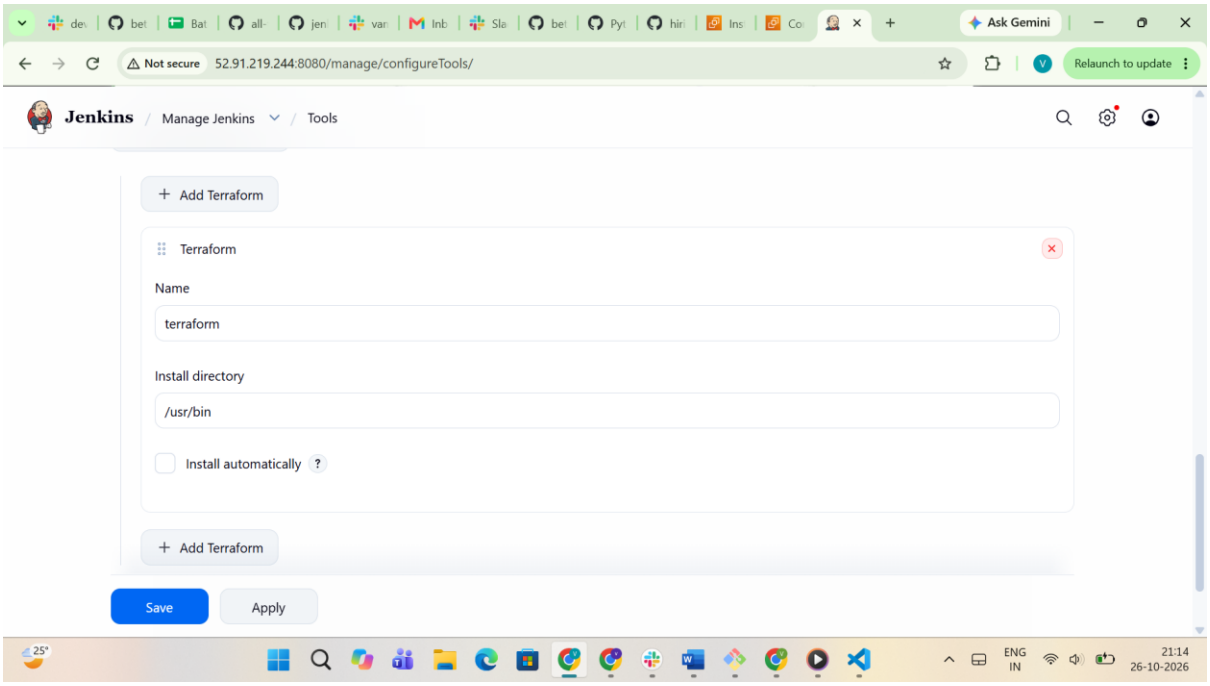
```
Installed:
  git-2.50.1-1.amzn2023.0.1.x86_64           git-core-2.50.1-1.amzn2023.0.1.x86_64 git-core-doc-2.50.1-1.amzn2023.0.1.noarch    perl-Error-1:0.17030-2.amzn2023.0.1.noa
  perl-File-Find-1.37-477.amzn2023.0.10.noarch perl-Git-2.50.1-1.amzn2023.0.1.noarch perl-TermReadKey-2.38-9.amzn2023.0.3.x86_64 perl-lib-0.65-477.amzn2023.0.10.x86_64
  terraform-1.17.0-1.x86_64

Complete!
[root@ip-172-31-20-222 ~]# terraform version
Terraform v1.17.0
on linux_amd64

[root@ip-172-31-20-222 ~]# which terraform
/usr/bin/terraform
[root@ip-172-31-20-222 ~]# sudo -u jenkins terraform version
Terraform v1.17.0
on linux_amd64

[root@ip-172-31-20-222 ~]# readlink -f "$(which terraform)"
/usr/bin/terraform
[root@ip-172-31-20-222 ~]# dirname "$(readlink -f "$(which terraform)")"
/usr/bin
[root@ip-172-31-20-222 ~]# ls -l /opt/terraform
ls: cannot access '/opt/terraform': No such file or directory
[root@ip-172-31-20-222 ~]# ^C
[root@ip-172-31-20-222 ~]# cd /usr
[root@ip-172-31-20-222 usr]# ll
total 176
dr-xr-xr-x.  2 root root 32768 Oct 23 10:44 bin
drwxr-xr-x.  2 root root     6 Jan 30  2023 games
drwxr-xr-x.  3 root root    23 Oct 21 17:47 include
dr-xr-xr-x. 32 root root 16384 Oct 23 10:31 lib
dr-xr-xr-x. 44 root root 32768 Oct 23 10:31 lib64
drwxr-xr-x. 27 root root 16384 Oct 23 10:44 libexec
drwxr-xr-x. 12 root root   131 Oct 21 17:47 local
dr-xr-xr-x.  2 root root 16384 Oct 21 17:49 sbin
drwxr-xr-x. 98 root root 16384 Oct 23 10:44 share
drwxr-xr-x.  4 root root    34 Oct 21 17:47 src
lrwxrwxrwx.  1 root root    10 Jan 30  2023 tmp -> ../var/tmp
[root@ip-172-31-20-222 usr]# ^C
[root@ip-172-31-20-222 usr]# sudo -u jenkins terraform version
Terraform v1.17.0
on linux_amd64
```





4. **Create one Jenkins job using Maven Project for the code below with two stages:**
    - **Stage 1: Git clone**
    - **Stage 2: Maven Compilation Code: https://github.com/betawins/java-Working-app.git**

Implementations steps:

1. Log in to Jenkins.
2. Click New Item.
3. Enter the job name Java-Working-App.
4. Select Pipeline and click OK.
5. Scroll down to the Pipeline section.
6. Select Pipeline script.
7. Enter the following script:

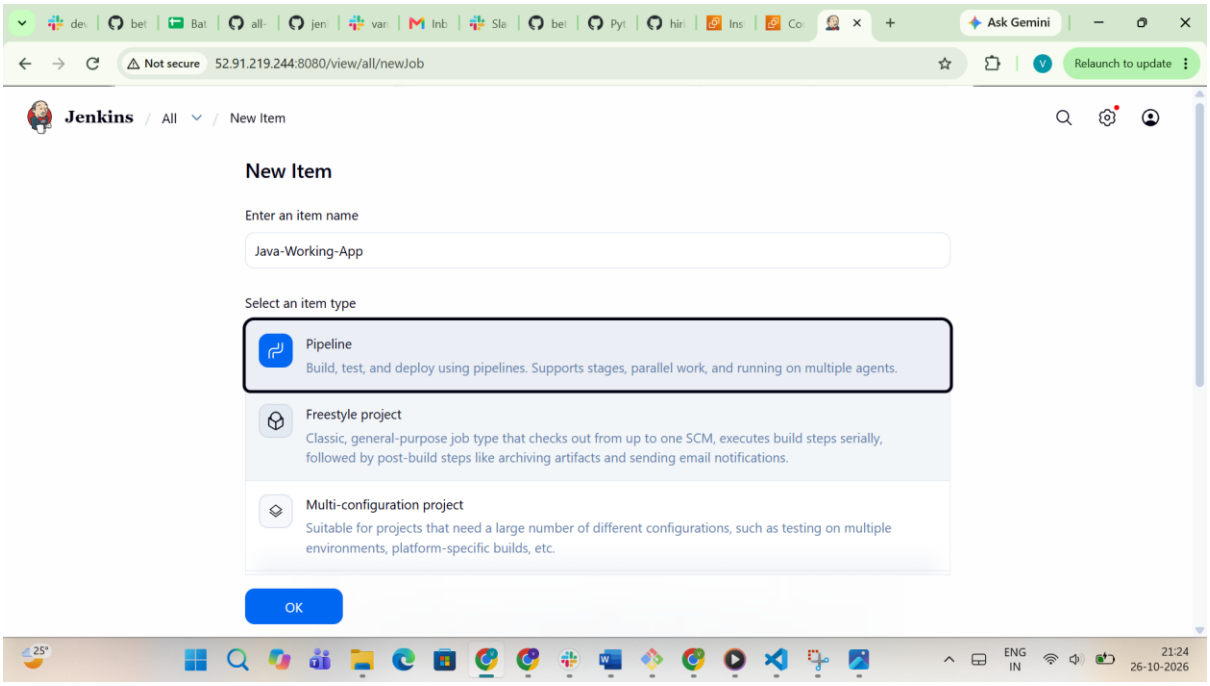
```
pipeline {
    agent any
```

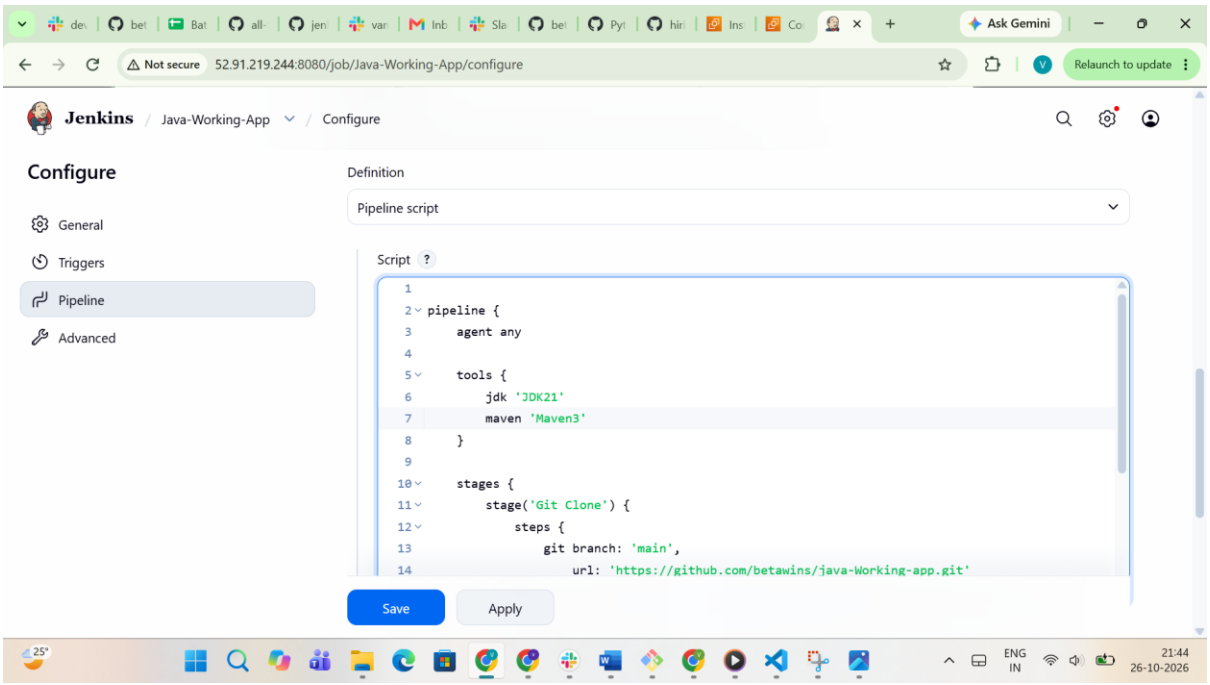
```
    tools {
        jdk 'JDK21'
        maven 'Maven3'
    }

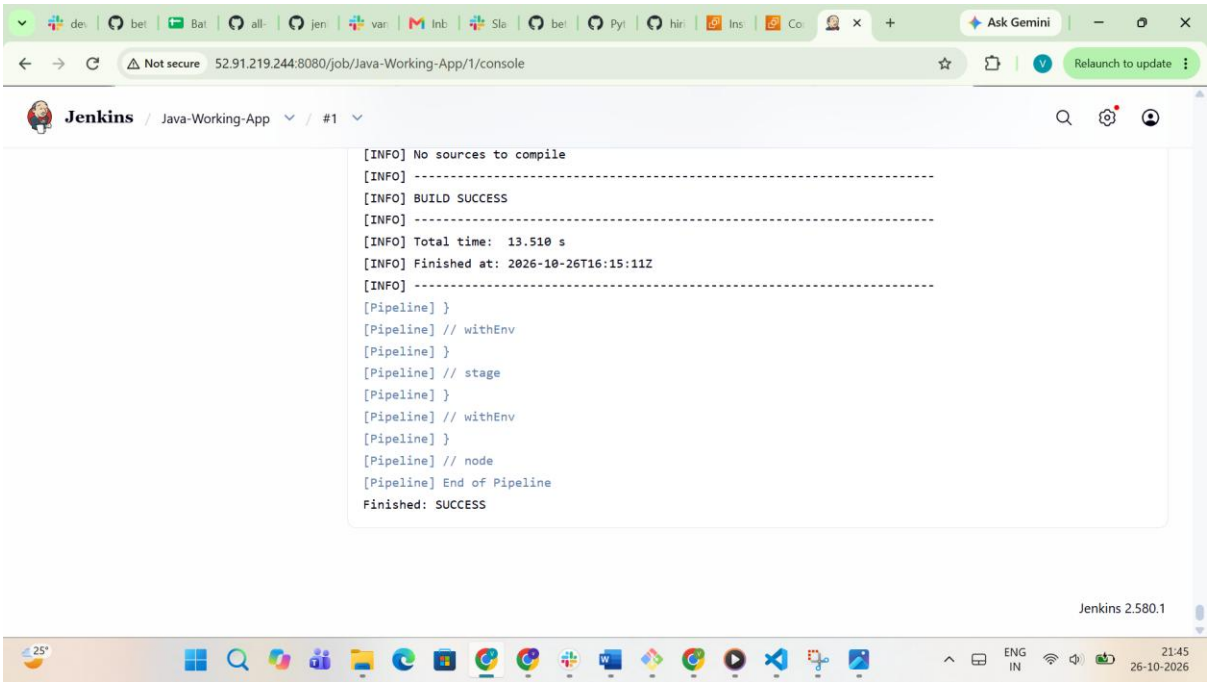
    stages {
        stage('Git Clone') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/betawins/java-Working-app.git'
            }
        }

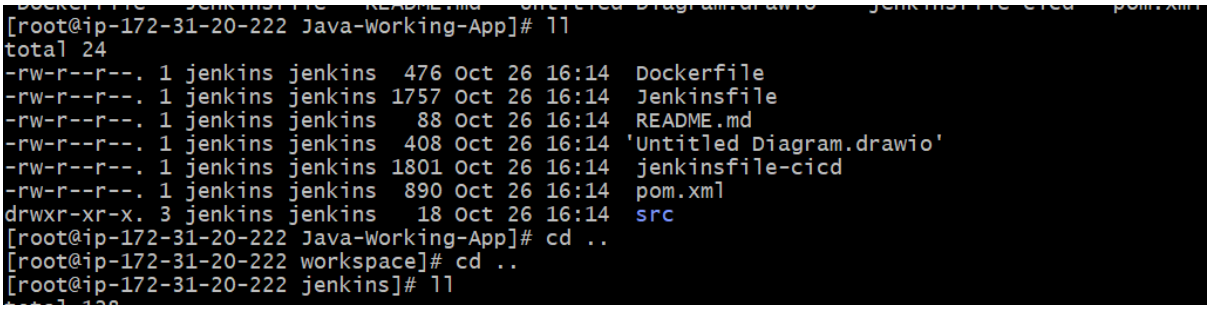
        stage('Maven Compilation') {
            steps {
                sh 'mvn clean compile'
            }
        }
    }
}
```

8. Ensure JDK21 and Maven3 match the names configured under Manage Jenkins → Tools.
9. Click Save.
10. Click Build Now.
11. Open Console Output and verify that both stages complete successfully.


Not secure
52.91.219.244:8080/view/all/newJob
Relaunch to update
Jenkins
All
New Item
New Item
Enter an item name
Java-Working-App
Select an item type
Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
OK


Not secure
52.91.219.244:8080/job/Java-Working-App/configure
Relaunch to update
Jenkins
Java-Working-App
Configure
Configure
General
Triggers
Pipeline
Advanced
Definition
Pipeline script
Script
pipeline {
agent any
tools {
jdk 'JDK21'
maven 'Maven3'
}
stages {
stage('Git Clone') {
steps {
git branch: 'main',
url: 'https://github.com/betawins/java-Working-app.git'
Save
Apply





5. **Use the same code and create a parameterized job in Jenkins with:**
    - **Stage 1: Git clone**
    - **Stage 2: Maven Compilation Code: https://github.com/betawins/java-Working-app.git**

Objective: Create a parameterized job that accepts a Git branch as input and performs Git checkout and Maven compilation.

Implementations steps:

1. Click New Item.
2. Enter the job name Java-Working-App-Parameterized.
3. Select Pipeline and click OK.
4. Under Pipeline, select Pipeline script.
5. Enter the following script:

```
pipeline {
    agent any

    parameters {
        string(
            name: 'GIT_BRANCH',
            defaultValue: 'main',
            description: 'Enter the Git branch name'
        )
    }

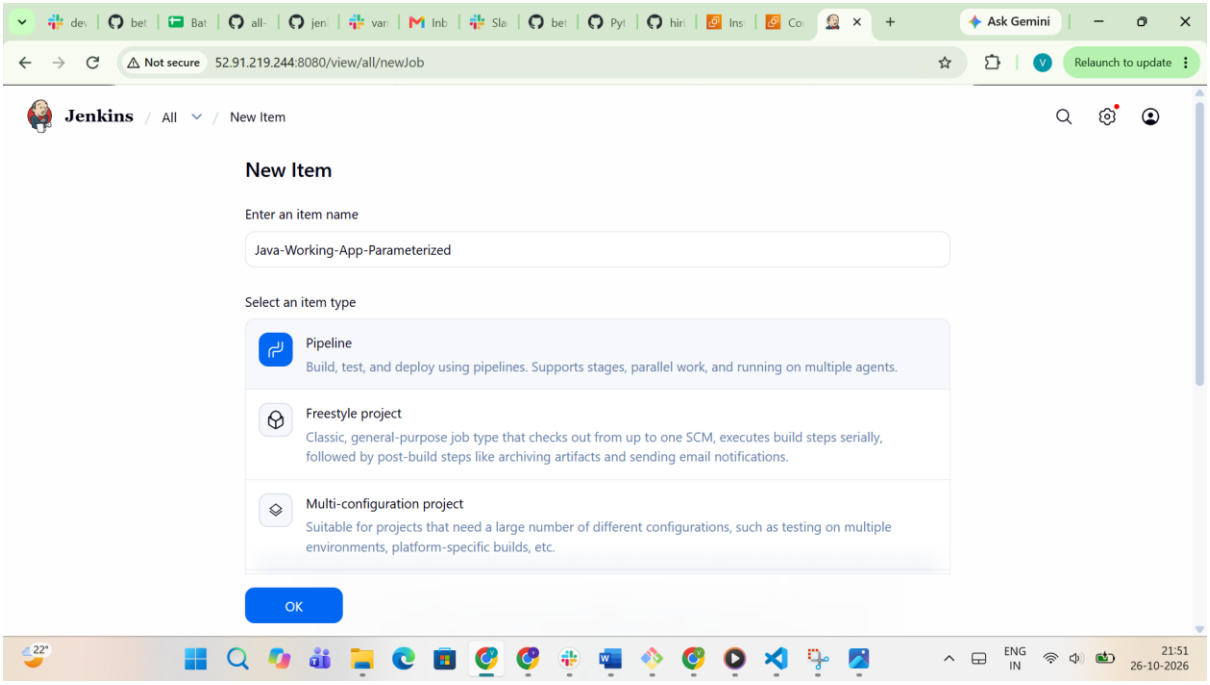
    tools {
        jdk 'JDK21'
        maven 'Maven3'
    }

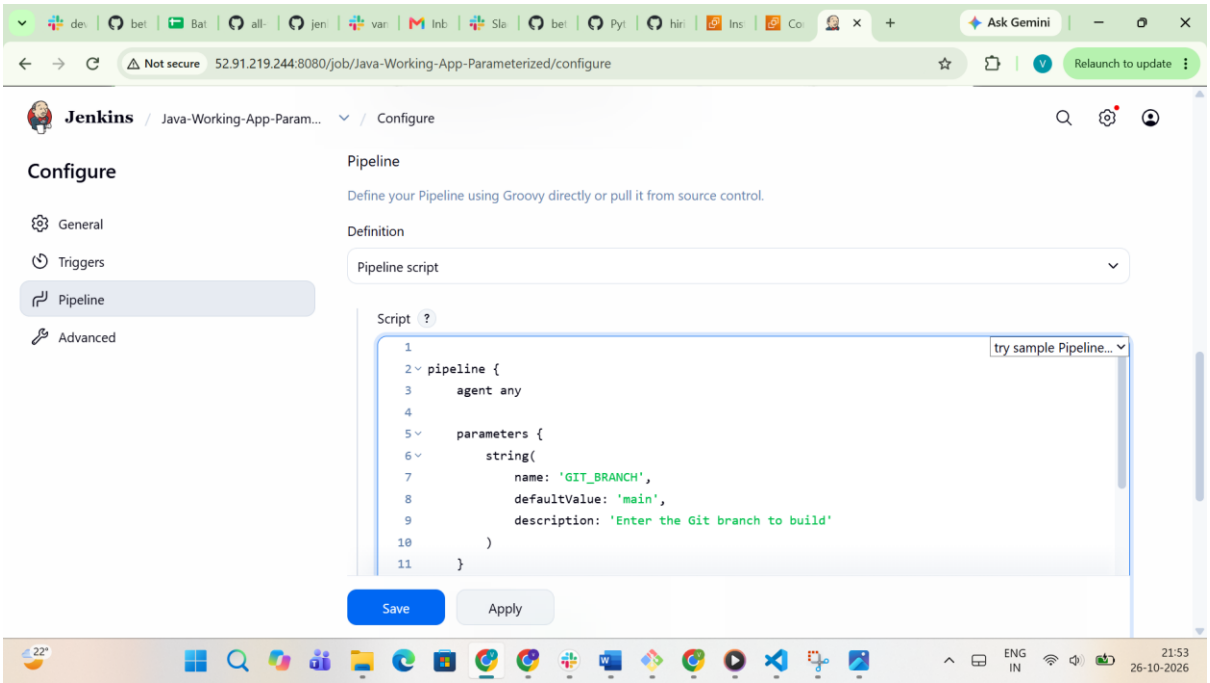
    stages {
        stage('Git Clone') {
            steps {
                git branch: params.GIT_BRANCH,
                    url: 'https://github.com/betawins/java-Working-app.git'
            }
        }

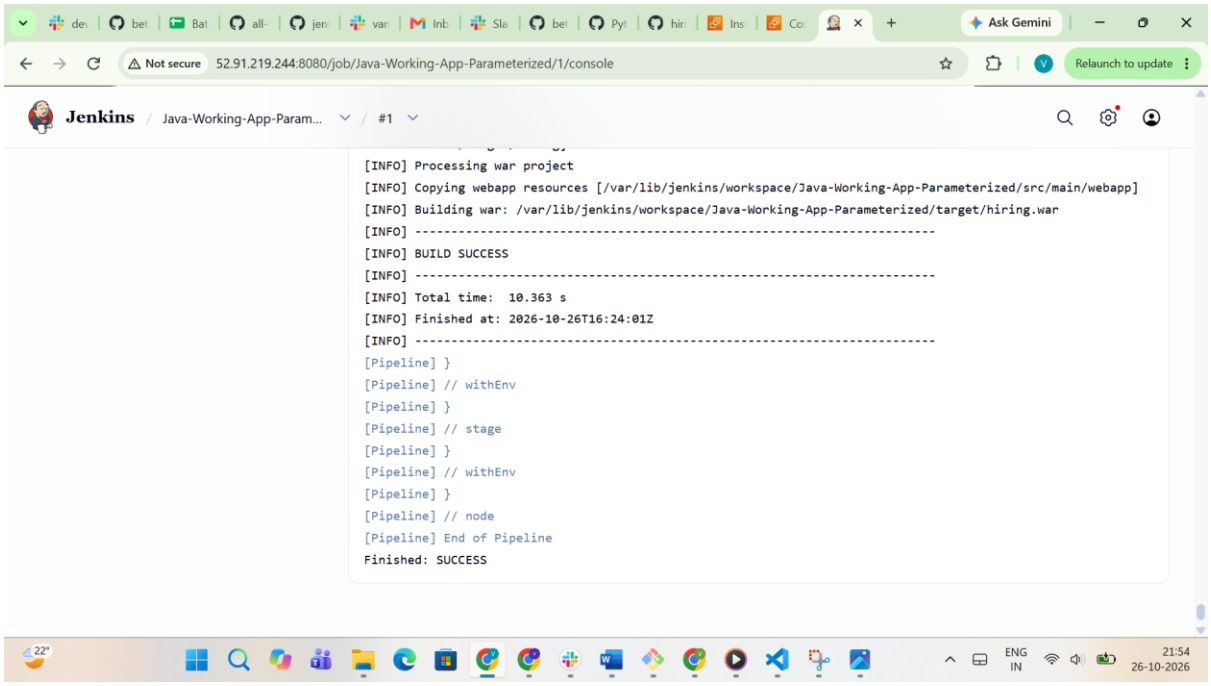
        stage('Maven Compilation') {
            steps {
                sh 'mvn clean compile'
            }
        }
```

```
    }
}
```

6. Click Save.
7. Click Build with Parameters.
8. Enter the Git branch name, such as main.
9. Click Build.
10. Verify both stages in Console Output.








```
[root@ip-172-31-20-222 Java-Working-App-Parameterized]# ll
total 24
-rw-r--r--. 1 jenkins jenkins  476 Oct 26 16:23  Dockerfile
-rw-r--r--. 1 jenkins jenkins 1757 Oct 26 16:23  Jenkinsfile
-rw-r--r--. 1 jenkins jenkins   88 Oct 26 16:23  README.md
-rw-r--r--. 1 jenkins jenkins  408 Oct 26 16:23 'Untitled Diagram.drawio'
-rw-r--r--. 1 jenkins jenkins 1801 Oct 26 16:23  jenkinsfile-cicd
-rw-r--r--. 1 jenkins jenkins  890 Oct 26 16:23  pom.xml
drwxr-xr-x. 3 jenkins jenkins   18 Oct 26 16:23  src
drwxr-xr-x. 4 jenkins jenkins   60 Oct 26 16:24  target
[root@ip-172-31-20-222 Java-Working-App-Parameterized]# cd target/
[root@ip-172-31-20-222 target]# ll
total 4
drwxr-xr-x. 4 jenkins jenkins   54 Oct 26 16:24 hiring
-rw-r--r--. 1 jenkins jenkins 1698 Oct 26 16:24 hiring.war
drwxr-xr-x. 2 jenkins jenkins   28 Oct 26 16:24 maven-archiver
[root@ip-172-31-20-222 target]#
```

6. What are the global variables in Jenkins?

Definition: Global variables in Jenkins are predefined variables or objects that provide information about the current job, build, workspace, parameters, and build status. They can be used in Pipeline scripts to access Jenkins information.

Common Jenkins variables

Variable	Purpose
env.BUILD_NUMBER	Current build number
env.BUILD_ID	Current build identifier
env.JOB_NAME	Name of the Jenkins job
env.BUILD_TAG	Unique identifier for the build
env.WORKSPACE	Directory where the job executes
env.JENKINS_URL	Jenkins URL, when configured
params	Accesses parameters supplied to the job

currentBuild	Provides information about the current build
`scm`	Represents SCM configuration in supported Pipeline contexts